

Docker

Docker

Les services

[A la découverte de docker-compose](#)

[Installation de docker-compose](#)

[Création de fichiers docker-compose.yml](#)

[Déploiement de services](#)

Les services

A la découverte de docker-compose

Docker Compose est un utilitaire fournit par Docker pour simplifier le déploiement d'application multi-conteneurs.

À partir d'un fichier déclaratif YAML `docker-compose.yml`, il déploie un service composé de plusieurs conteneurs.

Le nom du répertoire détermine le nom du service.

Les conteneurs sont désormais gérés par la commande `docker-compose`.

Documentation officielle : <https://docs.docker.com/compose/>

Installation de docker-compose

<https://docs.docker.com/compose/install/>

Vérifier l'installation :

```
docker-compose version
```

Création de fichiers docker-compose.yml

```
version: '3.7'
services:
  gitea:
    image: gitea/gitea:latest
    environment:
      - DB_TYPE=postgres
      - DB_HOST=db:5432
      - DB_NAME=gitea
      - DB_USER=gitea
      - DB_PASSWD=gitea
    restart: always
    volumes:
      - git_data:/data
    ports:
      - 3000:3000
  db:
```

```

image: postgres:alpine
environment:
  - POSTGRES_USER=gitea
  - POSTGRES_PASSWORD=gitea
  - POSTGRES_DB=gitea
restart: always
volumes:
  - db_data:/var/lib/postgresql/data
expose:
  - 5432
volumes:
  db_data:
  git_data:

```

Références du compose file : <https://docs.docker.com/compose/compose-file/compose-file-v3/>

Gisement d'exemples:

<https://docs.docker.com/compose/samples-for-compose/>

Exercice

Créer un `docker-compose.yml` pour l'exercice avec "Adminer" :

A l'aide de l'image `adminer` et en créant un réseau, déployez une interface Web pour gérer les bases de données de deux conteneurs `MariaDB` (à déployer également).

Déploiement de services

Les commandes

```

# builder et démarrer les conteneurs en mode détaché
docker-compose up -d
# --no-recreate ou --force-recreate

# démarrer les conteneurs
docker-compose start

# arrêter les conteneurs
docker-compose stop

# supprimer les conteneurs
docker-compose rm

# supprimer les conteneurs du service et les volumes
docker-compose down -v

# afficher les processus
docker-compose ps

# contrôler la syntaxe du fichier docker-compose.yml
docker-compose config

# afficher les logs
docker-compose logs -f

# augmenter le nombre de réplicats d'un service
docker-compose up -d --scale monservice=2

```

Référence des commandes : <https://docs.docker.com/compose/reference/>

Exercice

A l'aide de `docker-compose` déployez un ensemble de conteneurs avec une base de données MariaDB, une interface d'administration pour cette base de données (PhpMyAdmin) et un serveur Web avec PHP.

Il faudra vérifier que PhpMyAdmin parviens bien à joindre la base de données et que le fichier `index.php` suivant peut être exécuté sur le serveur WEB :

```
<?php
phpinfo();
?>
```

Exercice

Déployez le code `app.py` dans un conteneur.

Cette application gère son cache avec `Redis`.

Astuce : installez les dépendances `Python` avec la commande `pip install -r requirements.txt`

`app.py`

```
import time
import socket
import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
    retries = 5
    while True:
        try:
            return cache.incr('hits')
        except redis.exceptions.ConnectionError as exc:
            if retries == 0:
                raise exc
            retries -= 1
            time.sleep(0.5)

@app.route('/')
def hello():
    count = get_hit_count()
    name = socket.gethostname()
    ip = socket.gethostbyname(name)
    return f"Hello World! I have been seen {count} times. \n Generated by container {name} ({ip})"

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)
```

`requirements.txt`

